

REST API

Selecting the Connection Type

Once you have [selected your connection type](#), a configuration tab will appear on the next step, prompting you to input the required connection details.

Integration Setup

The screenshot shows the 'Add Target' configuration interface for a REST API connection. The interface is divided into three steps: 1. Select your connection, 2. Configure your connection (active), and 3. Configure your target. The configuration fields include:

- Connection Type:** REST API
- Send Message:** URL* (https://api.example.com)
- POST Request Payload Sample*:** A JSON payload with placeholders:

```
{  "message": "{message}",  "session_id": "{session_id}"}
```
- Response Path*:** choices[0].message.content
- Open Session:** Toggle switch (off)
- Close Session:** Toggle switch (off)
- OAuth:** Toggle switch (off)
- HTTP Headers:** Enter custom headers. A table shows a header with Key 'Authorization' and Value 'Bearer jwt-token'. A 'Sensitive' toggle switch is also present.

At the bottom right, there are 'Back' and 'Continue' buttons. The footer indicates '© 2025 SploitAI Inc. All rights reserved. Version 1.37.1-rc.9'.

Figure 1: Simple REST API Configuration Without Headers

- **URL** - This is your target endpoint to which the attack messages will be sent.
- **POST Request Payload Sample** - Here, you provide the payload (body of the HTTP request). Once provided, the payload should include the following placeholders, which you need to insert:
 - **{message}** - This placeholder represents where the Probe will insert attack messages, simulating input from a user interacting with your application.

- **{session_id}** - This placeholder marks the location where a unique string, identifying the current conversation session, will be placed. This ensures that the request is tied to a specific session for multi-step testing.
- The payload can contain additional fixed arguments if needed.
- **Response Path** - The JSON path pointing to the message within your chatbot's **API response** to the given request.
- **HTTP Headers** - Enter the key-value pairs necessary for your API Requests. Authorization headers must be included for non-public APIs (alternatively, [OAuth](#) can also be used).

 For HTTP header customization, the **Add Header +** button needs to be pressed after the Key and Value textboxes are filled.

✓ Obtaining the values

One way to obtain these values is by interacting with your application and inspecting the network requests using developer tools (e.g., in your browser or API testing tool like Postman).

- **URL:** Locate the endpoint URL where your chatbot sends requests. This is typically found in the network tab of developer tools or in your API documentation.
- **HTTP Headers:** Review the headers in the network request to identify any required key-value pairs, such as authorization tokens or content types.
- **Request Payload:** Copy the body of the POST request, then replace the user message with the **{message}** placeholder and the session identifier with the **{session_id}** placeholder.
- **Response Path:** Inspect the chatbot's API response and identify the JSON path to the specific part of the response where the chatbot's message is returned.

Session Management

Sometimes, your application may use different endpoints to manage sessions that track messages in conversations. These endpoints can be separate from the ones used for sending messages. For example:

- A session might be initiated with one request to an "open session" endpoint.
- The session might then be closed with another request to a "close session" endpoint.

If your application operates this way, the **Open Session** and **Close Session** options can be toggled and configured in the integration settings. This allows you to add the appropriate requests for starting and ending sessions, ensuring the Probe can properly simulate and test multi-step conversations.

OAuth

OAuth support for REST API connection is available to allow a third-party authentication **without sharing the user's credentials** (like username and password). To enable this method of authorization, you need to provide the following fields:

- **URL** - The endpoint of the OAuth authentication server (typically the token endpoint) used to request the access token (e.g. `https://auth.example.com/oauth/token`).
- **Client ID** - A unique identifier for the client application, provided by the OAuth server during app registration.
- **Client Secret** - A confidential key used by the application to authenticate itself to the OAuth server, used in combination with the Client ID.
- **Scope** - A space-separated list of permissions that the application is requesting, defining the level of access to protected resources.

These parameters are required to successfully authenticate and authorize access via OAuth. Make sure to obtain the correct values from your OAuth provider and configure them accordingly in the connection settings.

The screenshot displays the 'Add Target' configuration window in the AI RED TEAMING application. The interface is dark-themed with a sidebar on the left containing navigation links for PROBE, REMEDIATION, and MONITORING. The main panel shows a three-step process: 1. Select your connection, 2. Configure your connection (active), and 3. Configure your target. The 'Configure your connection' step includes fields for 'Response Path*' (choices.[0].message.content), 'Open Session' (toggle), 'Close Session' (toggle), 'OAuth' (toggle), 'URL*' (https://api.example.com), 'Client ID*' (placeholder: Place your client ID here), 'Client Secret*' (placeholder: Place your client secret here), and 'Scope*' (openid profile email). Below these is an 'HTTP Headers' section with a table for 'Key*', 'Value*', and 'Sensitive' status. The table contains one entry: 'Authorization' for 'Bearer jwt-token' with the 'Sensitive' toggle turned on. An 'Add Header +' button is at the bottom right of the table. At the very bottom of the form are 'Back' and 'Continue' buttons. The footer of the sidebar shows the copyright notice: © 2025 SpixAI Inc. All rights reserved. Version 1.37.1 rc.9.

Figure 2: OAuth Form

SPLX Proxy

API endpoints can be implemented in various ways, and we can't cover every scenario. To address this, we've developed the SPLX **Proxy Interface**, which you can use for easier integration.

Feel free to contact us, and we'll assist you in creating it.